# Pearls AQI Predictor

## End-to-End Air Quality Index Forecasting Platform

Final Project Report

**Muhammad Munawar Khan**

Software Engineering and Machine Learning Project

Prepared for project review and final submission

September 2026

# Document Control

| | |
|---|---|
| **Document title** | Pearls AQI Predictor: End-to-End Air Quality Index Forecasting Platform |
| **Author** | Muhammad Munawar Khan |
| **Document type** | Final project report |
| **Version** | 1.0 |
| **Submission status** | Final submission draft |
| **Primary audience** | Project managers, technical reviewers, and academic assessors |
| **Source specification** | AQI_Predictor_report.pdf |

**Scope note.** This report expands the supplied end-to-end implementation specification into a formal project document. It presents the intended architecture, data contract, machine-learning methodology, operational design, quality plan, risks, and acceptance criteria. Where the source specification does not provide executed experiment logs, measured model scores, or deployment evidence, this report explicitly labels those items as **verification pending** rather than inventing results.

# Abstract

The Pearls AQI Predictor is an end-to-end software and machine-learning platform designed to forecast the Air Quality Index (AQI) for a configured city over the next three days. The proposed system combines external pollutant and weather data, a canonical observation schema, time-series feature engineering, a local or cloud-compatible feature store, reproducible model training, a model registry, a Flask API, and a Streamlit dashboard. It is intentionally designed to operate in two modes: a real-provider mode for connected deployments and a deterministic demo mode for evaluation, testing, and development without external credentials.

The project addresses a practical communication problem: raw pollutant measurements are difficult for non-specialist users to interpret, while a short-horizon forecast can support planning and early awareness. The platform therefore does more than produce numeric predictions. It exposes data freshness, pollutant context, model metadata, feature influence, quality flags, and configurable alerts. The design emphasizes reproducibility and operational safety through UTC timestamps, validation, retry policies, chronological evaluation, leakage prevention, model versioning, structured logging, and explicit disclaimers.

The report documents the system from a project-management perspective. It defines the objectives and success criteria, explains the architecture and data flow, specifies the feature and target design, compares candidate model families, describes the dashboard and API contract, and establishes a testing and deployment plan. The AQI scale is treated as a configured reporting scale rather than an unquestioned universal standard because providers may use different conventions. Forecasts are engineering estimates and must be compared with official environmental information before they are used for public-health decisions.

**Keywords:** air quality, AQI forecasting, time-series machine learning, feature store, Streamlit, Flask, SHAP, data quality, model registry.

# 1 Executive Summary

The Pearls AQI Predictor is specified as a production-minded forecasting platform rather than a one-off notebook. Its value proposition is the integration of reliable data acquisition, disciplined machine-learning practice, and accessible communication in one coherent workflow. A user selects or configures a city, the system retrieves current or historical observations, transforms them into a validated feature set, generates a three-day AQI forecast, and presents the result together with context and uncertainty indicators.

The source specification defines a Python-first implementation with a provider abstraction, a local Parquet/SQLite-compatible feature store, scikit-learn models, an optional TensorFlow sequence model, a Flask service layer, and a Streamlit user interface. The implementation is expected to remain usable in demo mode so that missing credentials or unavailable cloud services do not block evaluation. This is a strong engineering choice because it separates core functionality from infrastructure dependencies and makes the project reproducible on a clean machine.

The project is organised around five outcomes: a complete prediction workflow; automated hourly and daily pipelines; an interactive dashboard; reproducible model training and evaluation; and documentation that a new contributor can follow. The success of the project is therefore not judged by a single accuracy value. It is judged by whether the system can move data safely through ingestion, validation, feature generation, training, evaluation, registration, inference, and presentation.

| Outcome | What the project must demonstrate | Evidence status |
|---|---|---|
| End-to-end forecast | Current or demo observations can be transformed into three future daily AQI values. | Specified; execution evidence to be attached after implementation. |
| Reproducible ML | Chronological split, candidate comparison, metrics, and model registration are repeatable. | Specified; measured scores require an executed run. |
| Operational interface | Flask endpoints and Streamlit views expose results, quality, alerts, and explanations. | Specified; runtime verification required. |
| Automation | Hourly feature updates and daily retraining are defined and manually runnable. | Specified in workflow design. |
| Governance | Secrets, provider assumptions, data limitations, and non-authoritative use are documented. | Included in this report. |

The most important management conclusion is that the project has been designed with a clear path from prototype to maintainable service. The local backend supports immediate demonstration; the abstract provider and feature-store interfaces preserve an upgrade path to AQICN, OpenWeather, Hopsworks, Vertex AI, or another compatible service. The use of a model registry protects inference from temporary retraining failures because the last known valid champion can remain available.

# 2 Table of Contents

# 3 1. Introduction

## 3.1 1.1 Background and motivation

Air-quality information is usually delivered as a current measurement, while planning decisions often require an expectation of what conditions may be like later. A forecasting system can connect historical observations, meteorological conditions, and recent pollutant behaviour to produce a short-horizon estimate. The AQI is particularly suitable for communication because it compresses multiple pollutant measurements into a scale that can be interpreted by non-specialist users. AirNow describes the U.S. AQI as an index for reporting outdoor air quality, with six categories ranging from Good to Hazardous [1].

The challenge is not merely selecting a regression algorithm. A useful forecasting product must handle inconsistent providers, missing values, time zones, data freshness, provider-specific AQI scales, model drift, and the distinction between a prediction and an official public-health statement. The Pearls AQI Predictor addresses these challenges by treating the project as a complete data product. The system includes data contracts, feature lineage, operational workflows, explanatory outputs, and explicit limitations.

## 3.2 1.2 Problem statement

The project problem can be stated as follows: **Given the latest validated air-quality and weather observations for a configured location, estimate the mean AQI for each of the next three calendar days and present the result in a transparent, reproducible, and operationally safe interface.**

This definition contains three important constraints. First, the output is multi-horizon: day one, day two, and day three are separate targets. Second, the system must use only information available at prediction time; future values must never enter the training features. Third, the output must be accompanied by data quality and model context so that users can distinguish a strong, fresh estimate from an uncertain result produced from stale or incomplete data.

## 3.3 1.3 Aim and objectives

The overall aim is to design and implement a deployable AQI forecasting platform that is understandable to both technical and non-technical stakeholders. The objectives are to establish a normalized provider-independent data model; implement reliable ingestion and validation; create leakage-safe time-series features; compare baseline and machine-learning models; register and serve the selected model; expose results through an API and dashboard; automate recurring workflows; and document installation, operations, testing, limitations, and future improvements.

## 3.4 1.4 Stakeholders and expected benefits

Project managers require clear milestones, acceptance criteria, and operational risks. Developers require modular interfaces and reproducible commands. Data scientists require a trustworthy dataset, time-aware evaluation, and feature lineage. End users require an interface that explains current conditions and forecast categories without requiring knowledge of model internals. Operations personnel require logs, health checks, retries, and recovery procedures.

The expected benefits are improved visibility into local air-quality trends, a repeatable forecasting workflow, a portfolio-quality demonstration of applied machine learning, and a foundation that can be connected to cloud services without redesigning the core domain logic.

## 3.5 1.5 Report organisation

The remainder of this report proceeds from system context to implementation design. Chapter 2 defines requirements and success criteria. Chapter 3 explains the architecture and technology choices. Chapter 4 describes data acquisition and governance. Chapter 5 documents feature engineering and target construction. Chapter 6 explains exploratory analysis and modelling. Chapter 7 covers evaluation and the model registry. Chapter 8 presents the API and dashboard. Chapter 9 discusses automation and operations. Chapter 10 defines the testing and quality strategy. Chapter 11 records security, risks, limitations, and future work. Appendices provide the data dictionary, command reference, and acceptance checklist.

# 4 2. Requirements and Success Criteria

## 4.1 2.1 Functional requirements

The system shall support current-data ingestion, historical backfill, normalized observation storage, feature computation, target construction, model training, model evaluation, model registration, three-day inference, explanation generation, alert classification, API access, and dashboard presentation. The system shall support both real API mode and deterministic demo mode. In demo mode, all core workflows must run without external credentials.

| ID | Requirement | Acceptance interpretation |
|---|---|---|
| FR-01 | Provider abstraction | AQICN, OpenWeather, and mock implementations can conform to the same current, historical, and forecast interface. |
| FR-02 | Canonical schema | Every accepted observation includes identity, timestamps, pollutant, weather, source, and quality fields. |
| FR-03 | Feature pipeline | Hourly processing produces leakage-safe lag, rolling, cyclical, change, and quality features. |
| FR-04 | Training | Naive, Ridge, and Random Forest candidates are trained using chronological data partitions. |
| FR-05 | Inference | The service returns three daily AQI forecasts, model metadata, freshness, and alert information. |
| FR-06 | Dashboard | The interface displays current AQI, pollutants, forecast, trends, explanations, data quality, and model performance. |
| FR-07 | Automation | Hourly feature and daily training workflows are scheduled and manually dispatchable. |
| FR-08 | Testing | Unit, data-contract, integration, smoke, and failure tests are included. |

## 4.2 2.2 Non-functional requirements

The platform should be reproducible, modular, observable, secure by default, and inexpensive to operate. Configuration must be externalised through environment variables and YAML or TOML. Timestamps should be stored internally in UTC. Secrets must not be committed or displayed in logs. The demo pipeline should use deterministic random seeds and should complete in a practical CI runtime. Errors should be structured and actionable rather than silently ignored.

## 4.3 2.3 Definition of done

The project is complete when a new user can create an environment, enable demo mode, run a short backfill, train and register a model, generate a forecast, start the Flask API, and open the Streamlit dashboard using documented commands. Completion also requires evidence of validation, evaluation, model metadata, dashboard behaviour, scheduled workflows, and test results. If a requirement is designed

but not executed in the submitted environment, the final handover must mark it as pending rather than claim completion.

### 4.4 2.4 Traceability approach

Each requirement is mapped to one or more implementation areas and one or more verification activities. This makes the project reviewable by a manager who may not inspect every source file. For example, FR-03 maps to the feature transformation module, the feature metadata document, leakage tests, and a pipeline smoke test. FR-05 maps to the prediction service, the API forecast endpoint, the dashboard forecast view, and an inference smoke test.

# 5 3. System Architecture

## 5.1 3.1 Architectural overview

The system follows a layered architecture. Provider adapters isolate external data formats. The canonical schema creates a stable boundary between acquisition and downstream processing. Validation and quality checks reject malformed records while retaining enough context for investigation. The feature layer computes model-ready variables and target values. The feature store provides a durable, queryable snapshot for training and inference. The model layer evaluates candidates and registers the champion. Finally, the prediction service is consumed by both the Flask API and the Streamlit dashboard.

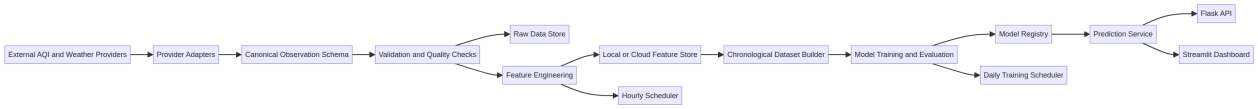


图 1 Logical architecture and data flow of the AQI Predictor.

## 5.2 3.2 Technology selection rationale

Python is the primary implementation language because it supports data acquisition, tabular transformation, machine learning, web APIs, testing, and dashboard development within one ecosystem. Scikit-learn provides practical regression, preprocessing, model-selection, and metric utilities [2]. Streamlit is appropriate for the primary interface because it is designed to deliver interactive Python data applications with limited front-end overhead [3]. Flask provides a lightweight HTTP service boundary. SHAP is selected for explainability because it frames feature attribution using Shapley-value concepts and supports model-specific and model-agnostic explainers [4].

The design intentionally avoids making both Hopsworks and Vertex AI mandatory. A `FeatureStoreClient` interface allows the local implementation to remain the default for development and demonstration, while cloud implementations can be introduced behind the same logical methods. This is a practical trade-off between immediate usability and future deployment flexibility.

| Layer | Technology | Responsibility |
|---|---|---|
| Language | Python 3.11+ | Core implementation, data processing, pipelines, API, dashboard, and tests. |
| Data and ML | Pandas, NumPy, scikit-learn | Normalization, features, preprocessing, candidate models, metrics. |

| | | |
|---|---|---|
| Optional deep learning | TensorFlow | Compact sequence model when data volume justifies it. |
| Storage | Parquet/SQLite local backend | Durable demo and test storage with a cloud-compatible abstraction. |
| API | Flask | Health, current, forecast, feature, model, and explanation endpoints. |
| Interface | Streamlit | Interactive operational and analytical dashboard. |
| Explainability | SHAP or fallback | Local feature contribution or importance summaries. |
| Automation | GitHub Actions; optional Airflow | Hourly feature refresh, daily training, CI, and artifacts. |

### 5.3 3.3 Repository organisation

The recommended repository separates configuration, source packages, dashboard code, scripts, tests, workflows, and documentation. This separation is more than stylistic: it reduces accidental coupling and makes ownership clear. Providers should not contain modelling logic. The dashboard should call a service boundary rather than recreate feature engineering. Tests should be grouped by responsibility so that a failure points to a likely layer.

### 5.4 3.4 Data-flow controls

The architecture contains four control points. The first is schema validation, which prevents malformed external data from silently entering the model. The second is temporal feature construction, which prevents future information from entering the training row. The third is chronological evaluation, which prevents random shuffling from overstating performance. The fourth is registry-based inference, which ensures the model and preprocessing artefacts are loaded as a versioned unit.

## 6 4. Data Acquisition and Governance

### 6.1 4.1 Provider abstraction

A common provider interface is required for current observations, historical observations, and forecasts. AQICN is the preferred pollutant/AQI provider in the source specification, while OpenWeather may supply weather variables or act as a compatible fallback. A mock provider creates deterministic records for demo execution. The abstraction ensures that downstream code receives normalized observations rather than provider-specific payloads.

The provider layer should implement timeouts, bounded retries with exponential backoff, rate-limit handling, response validation, structured logging, and a cached-data fallback where appropriate. Raw payloads should be preserved with source identifiers or hashes so that a later investigation can distinguish a provider change from a transformation error.

## 6.2 4.2 Canonical observation contract

The normalized observation schema is the central data contract. Identity fields identify the location and source. Time fields define the observation and retrieval moments. Pollutant fields represent AQI and major pollutant measurements. Weather fields represent meteorological context. Quality fields describe completeness, observation status, and quality flags.

| Group | Fields | Quality rule |
|---|---|---|
| Identity | location_id, city, country, latitude, longitude, source | Location is explicit and stable. |
| Time | observed_at, retrieved_at, timezone | Timestamps are timezone-aware; UTC is the internal standard. |
| Target | aqi | AQI scale is recorded and documented. |
| Pollutants | pm25, pm10, o3, no2, so2, co | Negative concentrations are rejected. |
| Weather | temperature_c, humidity_pct, pressure_hpa, wind_speed_ms, wind_direction_deg, precipitation_mm, cloud_pct | Reasonable physical ranges are validated. |
| Quality | is_observed, is_forecast, missing_field_count, data_quality_flag | Missingness is represented, not silently discarded. |

## 6.3 4.3 Validation and missing data

Missing provider fields must be represented as null until the transformation layer decides whether a feature can be computed. This distinction is important. A missing value is a property of the observation, whereas an imputed value is a property of the modelling decision. The system should record an imputation flag and preserve the original missing-field count. Imputation and scaling must be fitted on the training partition only.

Humidity should be between 0 and 100 percent. Pollutant concentrations should not be negative. Timestamps must be timezone-aware. Location identifiers should be sanitized. Duplicate records should be identified using location, observation time, and source. If a provider uses an AQI scale that differs from the configured display scale, the system must either convert it with documented logic or display the source scale explicitly.

## 6.4 4.4 Data governance principles

The platform should preserve source metadata, retrieval timestamps, units, transformation lineage, and quality status. These fields support auditability and help project managers judge whether an apparent model improvement is actually caused by a change in data coverage. Raw data should be retained outside the main source repository when it is large or sensitive. Secrets belong in environment variables or a managed secret store, never in code, notebooks, logs, or screenshots.

# 7 5. Feature Engineering and Target Design

## 7.1 5.1 Feature engineering principles

The feature pipeline converts irregular external observations into a structured time-series learning table. It begins with temporal normalization and sorting. It then creates calendar features, current measurements, lagged values, rolling statistics, change rates, and data-quality indicators. The pipeline must use past observations only. A feature that contains any information from the prediction interval is a leakage risk and must be excluded or explicitly justified.

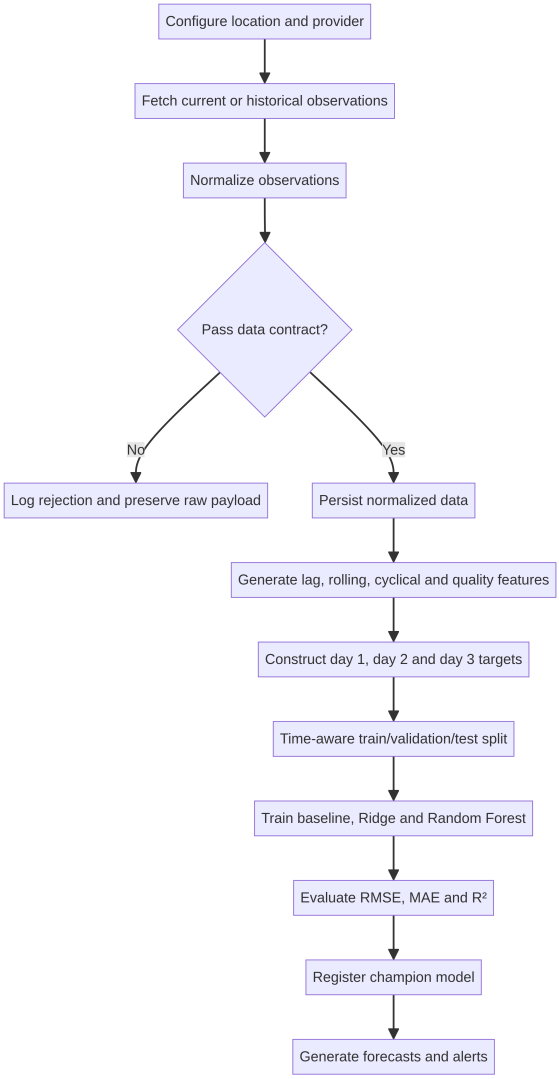


图 2 Operational sequence from data acquisition to forecast generation.

## 7.2 5.2 Required feature families

Calendar features include hour, day of week, day of month, month, week of year, weekend status, and cyclical encodings. Cyclical representations allow a model to understand that hour 23 and hour 0 are close in time rather than far apart numerically. Current pollutant and weather variables provide the latest state. Lags capture persistence and delayed effects. Rolling statistics capture local level and volatility. Change features represent direction and rate of movement. Quality features allow the model and dashboard to account for stale or incomplete observations.

| Family | Examples | Purpose |
|---|---|---|
| Calendar | hour, day_of_week, month, is_weekend, sin/cos hour and month | Represent recurring temporal patterns. |
| Current state | aqi_current, pm25, pm10, o3, no2, so2, co | Describe the latest observed condition. |
| Weather | temperature, humidity, pressure, wind, precipitation, cloud | Capture meteorological context. |
| Lags | 1h, 3h, 6h, 12h, and 24h for AQI and pollutants | Represent persistence and delayed effects. |
| Rolling | 3h, 6h, 12h, and 24h mean/min/max/std for AQI and PM2.5 | Represent local level and variability. |
| Change | aqi_change_1h, aqi_change_3h, aqi_change_rate_1h, pm25_change_1h | Represent momentum and recent deterioration. |
| Quality | missing_field_count, imputation_flag, stale hours, source reliability | Expose data confidence and freshness. |

## 7.3 5.3 Target definition

The default target is the mean AQI for each of the next three calendar days. For a reference timestamp, the pipeline constructs `target_aqi_day_1`, `target_aqi_day_2`, and `target_aqi_day_3`. A target is marked missing when insufficient observations exist for the relevant day. Training rows with missing targets are excluded, while the reason for exclusion is recorded in the pipeline report.

This formulation makes the forecasting objective explicit and consistent between training and inference. A model must receive the same feature semantics at runtime that it received during training. The prediction response should therefore include forecast dates, the model version, generation time, latest observation time, and data-quality status.

## 7.4 5.4 Leakage prevention

Leakage prevention is a primary quality control. The feature generator must shift lag and rolling calculations so that the current row does not include future observations. Dataset construction must sort by event time. Train, validation, and test partitions must follow chronological order. Preprocessing steps must be fitted only on training data. A leakage test should construct a synthetic sequence where a future value is changed and verify that earlier feature rows remain unchanged.

# 8 6. Exploratory Analysis and Modelling Methodology

## 8.1 6.1 Exploratory analysis plan

The EDA workflow should produce reproducible artefacts in a reports directory. It should quantify data completeness, distributions, trends, seasonality, pollutant correlation, AQI-category frequency, missingness, outliers, location differences, and the relationship between weather variables and AQI. EDA

is not a substitute for model evaluation; it is a diagnostic stage that identifies data problems and informs feature choices.

The analysis must distinguish training-period evidence from post-hoc test-period analysis. Decisions about feature definitions and model configuration should not be changed after observing test performance unless the evaluation protocol is restarted and the change is documented.

## 8.2 6.2 Dataset preparation

The dataset builder reads features and targets from the feature store, sorts rows by event time, removes invalid target rows, and applies a chronological split. The default split is 70 percent training, 15 percent validation, and 15 percent test, with a configurable temporal gap where necessary. Numerical imputation and scaling are fitted only on the training partition. The complete preprocessing pipeline is registered with the model so that inference uses the same transformation logic.

## 8.3 6.3 Candidate models

The naive persistence baseline predicts the latest known AQI. It is intentionally simple and establishes the minimum level of usefulness required from a machine-learning model. Ridge Regression provides a regularized linear baseline that is interpretable and efficient. Random Forest Regressor captures non-linear interactions and is a natural candidate for feature-importance explanations. A compact TensorFlow sequence model may be added if the historical dataset is large and stable enough to justify its training cost; it is not required merely to increase architectural complexity.

| Model | Strength | Primary limitation |
|---|---|---|
| Persistence baseline | Transparent reference point; extremely fast and robust. | Cannot learn weather or non-linear patterns. |
| Ridge Regression | Efficient, regularized, suitable for many correlated features. | Linear relationship may underfit complex interactions. |
| Random Forest | Captures non-linearity and interactions; supports importance analysis. | Larger artefact and less smooth extrapolation. |
| Sequence model | Can represent temporal sequences directly when data is sufficient. | Higher data, compute, and maintenance requirements. |

## 8.4 6.4 Model configuration and reproducibility

Model searches should use small, configuration-driven parameter grids. Random seeds must be fixed. Training duration, inference latency, data range, feature list, preprocessing version, source configuration, and available commit identifier should be stored with the model. The goal is not to search every possible hyperparameter. The goal is to create a defensible, repeatable comparison that can be rerun when data changes.

## 8.5 6.5 Metric selection

RMSE penalizes large errors more strongly and is useful when severe forecast misses are operationally costly. MAE is easier to interpret as an average absolute AQI error. $R^2$ expresses explained variance but should not be used as the sole selection criterion. Metrics should be reported separately for each horizon and overall. Category accuracy or macro-F1 may be added as a supplementary communication metric after AQI categories are defined.

# 9 7. Evaluation, Explainability, and Model Registry

## 9.1 7.1 Champion selection

The champion should be selected using a documented rule: lowest validation MAE, subject to acceptable test performance and a minimum data-quality threshold. This rule prevents a model from being selected solely because it has the highest $R^2$ on one split. If the champion is only marginally better than the baseline, that result should be reported honestly because operational simplicity may be preferable to a complex model with limited incremental benefit.

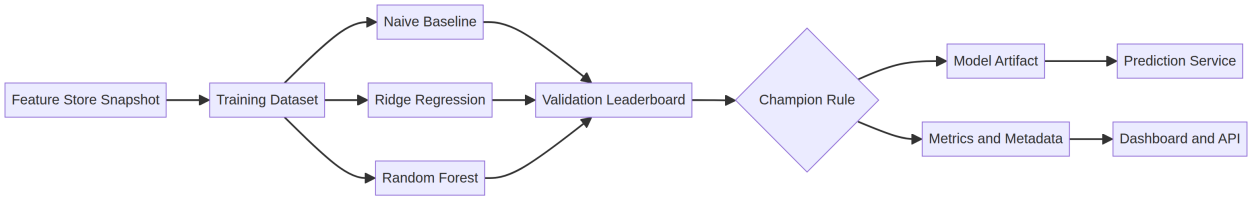


图 3 Model comparison, promotion, and inference lifecycle.

## 9.2 7.2 Required evaluation artefacts

The training pipeline should save a machine-readable metrics file, a human-readable leaderboard, prediction-versus-actual plots, and a model metadata record. Each record should include the dataset date range, number of training and test rows, target definitions, metrics by horizon, training duration, inference time, and model configuration. These artefacts make project review more efficient because a manager can inspect the decision without reading all implementation code.

## 9.3 7.3 Explainability strategy

For a Random Forest champion, SHAP values can provide local explanations for individual predictions. A fallback based on permutation importance or model coefficients should be available if SHAP is unavailable or unsuitable for the selected model. Explanations must be described as associations used by the model, not as causal claims. A feature that contributes positively to a prediction does not prove that changing that feature would cause AQI to rise.

The dashboard should present the most influential features in plain language, for example: recent AQI persistence, PM2.5 lag, humidity, wind speed, or a rolling volatility measure. The API should return structured feature-importance values so that the interface remains a presentation layer rather than a second explanation engine.

## 9.4 7.4 Model registry

The registry stores the serialized model, preprocessing artefacts, feature list, model name, version, training timestamp, data range, metrics, configuration, schema version, and optional source-control commit. It supports model registration, listing, loading by version, and champion selection. The registry is local by default but keeps a cloud-compatible interface. Production systems must define a trust boundary for serialized models and should never load an untrusted artefact.

## 9.5 7.5 Uncertainty and honest communication

Forecast intervals may be estimated from tree-ensemble dispersion, validation residuals, quantile models, or another documented method. If the interval is an approximation, the dashboard must label it as such. The presence of an interval does not guarantee calibrated probability. This distinction should be included in the user-facing disclaimer and in the limitations chapter.

# 10 8. Prediction Service, API, and Dashboard

## 10.1 8.1 Prediction service

The prediction service loads the champion model and preprocessing artefacts, reads the latest feature vector, generates three daily forecasts, clips only physically impossible negative outputs to zero, and attaches model metadata and freshness information. It should not silently retrain during an inference request. If the newest model is unavailable, the service should use the last known valid champion and emit a stale-model warning.

A representative response contains the configured location, generation timestamp, model name and version, three forecast objects, latest observation time, missing-feature count, and an alert object. This response is intentionally self-describing so that the dashboard, API consumers, and test fixtures can use the same contract.

## 10.2 8.2 Flask API contract

| Endpoint | Purpose |
|---|---|
| GET /health | Service health, version, timestamp, and status. |
| GET /api/v1/locations | Configured locations and identifiers. |
| GET /api/v1/current?location_id=… | Latest normalized observation and quality metadata. |
| GET /api/v1/forecast?location_id=… | Three-day forecast, categories, uncertainty, model, and alert. |
| GET /api/v1/features?location_id=… | Recent feature values for visualization and diagnostics. |
| GET /api/v1/model | Champion model metadata and metrics. |
| GET /api/v1/explanation?location_id=… | SHAP or fallback feature contributions. |

All responses should use consistent JSON error objects, HTTP status codes, request identifiers, and timestamps. Stack traces, filesystem paths, credentials, and internal secrets must not appear in user-facing responses. CORS should be enabled only when required by deployment.

## 10.3 8.3 Streamlit dashboard design

The dashboard should be organized around the questions a user asks. First, is the service working and how fresh is the data? Second, what is the current AQI and which pollutants are contributing context? Third, what is expected over the next three days? Fourth, why did the model produce this forecast? Fifth, how reliable is the underlying data and model?

The interface should include a location and status panel, current AQI and pollutant cards, weather context, a three-day forecast chart and table, historical trends, alert banners, explanation views, model-performance summaries, and data-quality indicators. Colour should reinforce meaning but never carry meaning alone; text labels and accessible contrast are required. The dashboard must state that forecasts are estimates and are not a substitute for official public-health guidance.

## 10.4 8.4 AQI categories and alerts

The system should support at least informational, moderate, warning, and hazardous alert states. When the selected reporting scale follows the U.S. AQI convention, the six common categories are Good (0–50), Moderate (51–100), Unhealthy for Sensitive Groups (101–150), Unhealthy (151–200), Very Unhealthy (201–300), and Hazardous (301+) [1]. Because a provider may use another scale, category mapping and thresholds must be configurable and documented.

# 11 9. Automation and Operations

## 11.1 9.1 Scheduled workflows

The hourly feature workflow retrieves current data, validates and normalizes it, computes features, updates the feature store, performs health checks, and retains logs and artefacts. The daily training workflow reads the latest valid training data, runs EDA checks, trains candidate models, evaluates them, registers the champion, and publishes metrics. GitHub Actions is a practical default for repository-based automation because workflows are visible, reviewable, and manually dispatchable [5]. Airflow may be added when task-level scheduling, retries, and dependency management become more demanding [6].

## 11.2 9.2 Operational controls

Each run should have a run identifier, pipeline name, location, provider, row counts, data freshness, latency, and error category. Transient provider failures should be retried with bounded exponential backoff. Malformed responses should be rejected and investigated rather than retried indefinitely. A failed training run should not remove the last known valid champion. The dashboard should show stale-data and stale-model indicators rather than presenting old values as current.

## 11.3 9.3 Backfill design

The backfill command accepts a location, start date, end date, provider, and batch interval. It must be restartable and idempotent. Checkpoints record completed and failed intervals. The final report includes the requested range, successful observations, rejected observations, missing intervals, target coverage, and final feature-store row count. Demo mode uses deterministic synthetic data, allowing the process to be tested without a provider account.

## 11.4 9.4 Deployment options

The local deployment path is the reference environment: a Python virtual environment, local feature store, local model registry, Flask development server, and Streamlit application. A containerized deployment can package the API and dashboard with environment-based configuration. A cloud deployment can replace the local feature store and scheduler while preserving the provider, model, prediction, and API interfaces. This staged approach reduces the risk of building a cloud-only design that cannot be evaluated locally.

## 11.5 9.5 Operational runbook

When data is stale, the operator should inspect provider status, rate-limit logs, and the last successful run before changing the model. When training fails, the operator should preserve the existing champion, review data-quality metrics, and rerun only after identifying the cause. When the dashboard fails, the operator should first check the API health endpoint and then inspect configuration and model-registry paths. When a model behaves unexpectedly, the operator should compare feature distributions and provider metadata before tuning hyperparameters.

# 12 10. Testing and Quality Assurance

## 12.1 10.1 Test strategy

Testing is organised by risk. Unit tests verify small transformations and business rules. Data-contract tests verify schema, types, timestamps, ranges, duplicates, and missing-field behaviour. Integration tests verify the hand-off between providers, feature store, training, registry, inference, and API. Smoke tests verify that the complete demo path works on a clean configuration. Failure tests verify that the system fails safely when an external dependency or internal artefact is unavailable.

| Test group | Coverage | Evidence |
|---|---|---|
| Unit | Features, AQI categories, target construction, validation, configuration, alerts, normalization. | Pytest results. |
| Data contract | Required columns, types, timezone, ranges, duplicates, missing fields. | Contract fixtures and assertions. |
| Integration | Mock provider to store; store to training; registry to inference; API endpoints. | End-to-end test logs. |
| Dashboard smoke | Application starts and primary data path loads in demo mode. | Startup and response check. |
| Pipeline smoke | Backfill, train, register, and three-day prediction. | Demo workflow output. |
| Failure | Timeout, malformed response, insufficient data, stale data, missing model, absent secret. | Expected error objects and safe fallback. |

## 12.2 10.2 Acceptance-test sequence

The recommended acceptance sequence is to install dependencies, copy the environment template, run unit and integration tests, run demo ingestion, run a short backfill, train candidate models, generate a forecast, start the API, query health and forecast endpoints, and launch the dashboard. The sequence should be executed on a clean configuration so that hidden local state does not mask missing setup steps.

## 12.3 10.3 Quality gates

A change should not be accepted if it introduces a failing test, changes a target definition without updating documentation, allows future data into a feature, exposes credentials, removes the last known valid model, or presents stale data without a warning. Model changes should include updated metrics and a short explanation of why the new champion was selected. Pipeline changes should include a data-contract or smoke-test update where relevant.

## 12.4 10.4 Verification status in this report

The supplied PDF is an implementation specification rather than a repository execution log. Therefore, this final report does not fabricate a test count, model leaderboard, API URL, or accuracy result. Those values should be filled from the actual execution artefacts after the project is run. This is a strength in a formal review: transparent evidence is more credible than invented precision.

# 13 11. Security, Risks, Limitations, and Future Work

## 13.1 11.1 Security and privacy

API keys and deployment credentials must be supplied through environment variables or a managed secret service. `.env`, credential files, generated model binaries, raw datasets, and logs should be excluded from version control when appropriate. Logs must not print secrets. Location identifiers supplied by users should be validated and sanitized. The API should expose a safe error object rather than a Python stack trace or internal filesystem path.

Serialized models create a trust-boundary concern. Only artifacts produced by a trusted training process should be loaded in production. Model metadata should be checked against the expected schema version and feature list. If an external model registry is introduced, access control and artifact integrity should be defined before production use.

## 13.2 11.2 Project risks

| Risk | Impact | Mitigation | Owner focus |
|---|---|---|---|
| Provider outage | Stale or missing observations. | Retries, cached fallback, stale indicators, mock mode. | Operations |
| AQI scale mismatch | Misleading categories or thresholds. | Record source scale; configure or document conversion. | Data engineering |

| | | | |
|---|---|---|---|
| Insufficient history | Unstable rolling features and weak model evidence. | Minimum-row checks; graceful skip of sequence model. | ML engineering |
| Temporal leakage | Overstated test performance. | Shifted features, chronological split, leakage tests. | Data science |
| Model drift | Forecast quality degrades over time. | Daily evaluation, monitoring, retraining, champion rollback. | ML operations |
| Credential exposure | Security incident or service abuse. | Secret management, safe logs, `.gitignore`, access control. | All contributors |

## 13.3 11.3 Known limitations

The forecast is limited by provider coverage, measurement quality, target availability, and the short history used for training. A three-day AQI estimate cannot fully capture unusual events such as wildfire smoke, dust storms, industrial incidents, abrupt weather changes, or provider outages unless those signals are represented in the data. Uncertainty intervals may be approximate. A model explanation is not causal evidence. AQI categories may differ by jurisdiction and provider. The system is an engineering and forecasting project, not an official environmental monitoring authority or a medical decision tool.

## 13.4 11.4 Future improvements

The next improvements should be prioritised by value and evidence. First, collect longer and more diverse historical data across locations. Second, add drift monitoring and calibrated prediction intervals. Third, compare direct multi-output models with separate horizon-specific models. Fourth, introduce richer meteorological and satellite features where data governance permits. Fifth, add a managed feature store and registry only when scale or collaboration requires them. Finally, add user feedback and incident logging so that dashboard warnings can be evaluated against operational outcomes.

# 14 12. Implementation and Handover Guide

## 14.1 12.1 Recommended implementation sequence

The implementation sequence begins with configuration, logging, schemas, and provider interfaces. The mock provider and canonical data contract should be implemented first so that all later work has a stable test fixture. Real provider adapters come next, followed by feature transformations, target construction, leakage checks, the local feature store, backfill, EDA artefacts, model training, evaluation, registry, inference, explanations, API, dashboard, automation, tests, and final documentation.

This order reduces rework because each stage builds on a verified interface. It also supports incremental project reviews. A manager can see meaningful progress after the data contract, after the end-to-end demo path, and after the first model comparison, rather than waiting for every cloud integration to be complete.

## 14.2 12.2 Developer commands

The following command set is the intended developer experience:

```
python -m venv .venv source .venv/bin/activate pip install -r requirements.txt
cp .env.example .env pytest -q python scripts/fetch_current.py –demo python scripts/
backfill.py –demo –start 2025-01-01 –end 2025-02-01 python scripts/train.py –demo python
scripts/predict.py –city Delhi –demo flask –app src.pearls_aqi.api.app run –debug streamlit
run dashboard/streamlit_app.py
```

## 14.3 12.3 Handover checklist

| No. | **Handover item** | **Status** |
|---|---|---|
| 1 | README explains setup, demo mode, real-provider mode, commands, tests, and deployment. | To verify |
| 2 | Architecture and data-flow documentation are present. | Specified |
| 3 | Data dictionary and feature metadata are present. | Specified |
| 4 | Operations and recovery procedures are present. | Specified |
| 5 | Model report includes EDA plan, split, metrics, and limitations. | To populate |
| 6 | API and dashboard can run in demo mode. | To verify |
| 7 | No secrets are committed or printed. | Required |
| 8 | Acceptance tests and execution logs are attached. | To verify |

## 14.4 12.4 Management conclusion

The Pearls AQI Predictor is a credible project because it connects a meaningful user problem with an explicit engineering lifecycle. It does not rely on a single model or a visually impressive dashboard alone. Instead, it defines how data is acquired, validated, transformed, evaluated, explained, served, monitored, and recovered. The design is suitably ambitious for a portfolio or academic project while remaining practical through demo mode and local defaults.

The most important recommendation for final submission is to attach execution evidence alongside this report: the test output, model leaderboard, generated forecast, screenshots of the dashboard, API responses, and the final repository commit. Once those artefacts are available, the verification placeholders in this report can be replaced with measured results without changing the architecture narrative.

# 15 Appendix A. Data Dictionary

| Field | Type | Unit | Meaning and null behaviour |
|---|---|---|---|
| location_id | string | — | Stable configured identifier; required. |
| observed_at | datetime | UTC | Measurement time; required and timezone-aware. |
| retrieved_at | datetime | UTC | Provider retrieval time; required for freshness. |
| aqi | float | index points | Observed AQI on documented source scale. |
| pm25 | float | µg/m³ or source unit | Fine particulate matter; nullable. |
| pm10 | float | µg/m³ or source unit | Coarse particulate matter; nullable. |
| o3, no2, so2, co | float | source unit | Major pollutants; nullable by provider. |
| temperature_c | float | °C | Air temperature; nullable. |
| humidity_pct | float | % | Relative humidity; validated 0–100. |
| pressure_hpa | float | hPa | Atmospheric pressure; nullable. |
| wind_speed_ms | float | m/s | Wind speed; non-negative. |
| precipitation_mm | float | mm | Precipitation amount; non-negative. |
| data_quality_flag | string | — | Freshness, completeness, and validation status. |

# 16 Appendix B. Configuration Reference

| Variable | Example | Purpose |
|---|---|---|
| APP_ENV | development | Runtime environment label. |
| DEMO_MODE | true | Use deterministic local provider and fixtures. |
| DEFAULT_CITY | Delhi | Default configured location. |
| TIMEZONE | UTC | Display and internal time policy. |
| DATA_PROVIDER | mock | Provider adapter selection. |
| AQICN_TOKEN | empty in demo | External provider credential; secret. |
| FEATURE_STORE_BACKEND | local | Local or cloud feature-store implementation. |
| FEATURE_STORE_PATH | data/features | Feature artefact location. |
| MODEL_REGISTRY_PATH | data/models | Model and metadata location. |
| FORECAST_HORIZON_DAYS | 3 | Number of future daily forecasts. |
| POLLUTION_ALERT_AQI | 150 | Configurable warning threshold. |
| HAZARDOUS_AQI | 300 | Configurable high-severity threshold. |
| MIN_TRAINING_ROWS | 168 | Minimum data threshold before training. |

# 17 Appendix C. Acceptance Checklist

| No. | **Acceptance criterion** | **Evidence** |
|---|---|---|
| 1 | Clean installation and demo mode run without external credentials. | Attach terminal log. |
| 2 | Raw and normalized observations are persisted with source metadata. | Attach sample artefact. |
| 3 | Features and targets are generated without future leakage. | Attach leakage test. |
| 4 | Naive, Ridge, and Random Forest models are compared using RMSE, MAE, and $R^2$. | Attach leaderboard. |
| 5 | Champion model and preprocessing artefacts are registered and reloadable. | Attach registry listing. |
| 6 | Three-day prediction command returns dates, AQI, model, freshness, and alert fields. | Attach JSON. |
| 7 | Flask health, current, forecast, model, and explanation endpoints respond correctly. | Attach API output. |
| 8 | Streamlit dashboard displays current AQI, forecast, trends, alerts, explanations, quality, and metrics. | Attach screenshots. |
| 9 | Hourly and daily workflows are present and manually runnable. | Attach workflow run. |
| 10 | Secrets are excluded and limitations are documented. | Code review. |

# 18 References

[1] [AirNow.gov, “Air Quality Index (AQI) Basics,”](https://www.airnow.gov/aqi/aqi-basics) accessed September 2026.
[2] [Scikit-learn, “Machine Learning in Python,”](https://scikit-learn.org/stable/) accessed September 2026.
[3] [Streamlit, “Streamlit Documentation,”](https://docs.streamlit.io/) accessed September 2026.
[4] [SHAP, “SHAP Documentation,”](https://shap.readthedocs.io/) accessed September 2026.
[5] [GitHub, “GitHub Actions Documentation,”](https://docs.github.com/en/actions) accessed September 2026.
[6] [Apache Airflow, “Apache Airflow Documentation,”](https://airflow.apache.org/docs/) accessed September 2026.
[7] [Flask, “Flask Documentation,”](https://flask.palletsprojects.com/) accessed September 2026.
[8] [AQICN, “AQICN API Documentation,”](https://aqicn.org/api/) accessed September 2026.
[9] [OpenWeather, “Weather API Documentation,”](https://openweathermap.org/api) accessed September 2026.
[10] [Hopsworks, “Hopsworks Documentation,”](https://docs.hopsworks.ai/) accessed September 2026.
[11] [Google Cloud, “Vertex AI Documentation,”](https://cloud.google.com/vertex-ai/docs) accessed September 2026.
[12] [TensorFlow, “TensorFlow Documentation,”](https://www.tensorflow.org/) accessed September 2026.

# 19 Declaration of Originality and Use

This report was prepared for the Pearls AQI Predictor project under the authorship of Muhammad Munawar Khan. It is based on the supplied project specification and expands that specification into a structured technical report. Any measured implementation results, screenshots, logs, or repository-specific values should be inserted from the final executed project artefacts before submission. The report intentionally distinguishes design commitments from verified runtime evidence.

The system described here is an engineering forecasting platform. Its outputs should be interpreted as estimates and compared with official environmental monitoring and public-health guidance. The project should not be represented as a medical diagnostic system, an official regulatory monitor, or a guarantee of future atmospheric conditions.

---

Muhammad Munawar Khan
Final Project Report
September 2026